POLITECHNIKA CZĘSTOCHOWSKA Katedra Informatyki	**Laboratorium z przedmiotu** **Projekt zespołowy**
KARTA PROJEKTU	
GRUPA: 2 Zespół: IO_2	Rok akademicki 2026/2027

1. Temat projektu:

Projektowanie i implementacja aplikacji webowej do zarządzania sklepem.

2. Kierownik projektu:

Yahor Hurski

3. Skład zespołu:

Imię i nazwisko	Główne zadanie
Yahor Hurski	Uwierzytelnianie i użytkownicy
Nazar Koretskyi	Katalog produktów
Stepan Tarasevych	Koszyk i zamówienia

4. Opis projektu:

Projekt obejmuje stworzenie aplikacji webowej typu sklep internetowy. Użytkownik będzie mógł przeglądać katalog produktów, wyszukiwać i filtrować produkty, założyć konto, zalogować się, dodawać produkty do koszyka oraz składać i przeglądać zamówienia.

Administrator będzie zarządzał produktami, kategoriami oraz statusami zamówień. Frontend zostanie wykonany w React, backend jako REST API w ASP.NET Core, a dane będą przechowywane w bazie PostgreSQL z użyciem Entity Framework Core.

Cały system będzie uruchamiany w kontenerach za pomocą Docker, co zapewni spójne środowisko pracy dla całego zespołu.

4.1 Cel projektu:

Celem projektu jest zaprojektowanie i wykonanie kompletnej, konteneryzowanej aplikacji sklepu internetowego działającej w architekturze klient–serwer.

Projekt ma umożliwić praktyczne wykorzystanie React, REST API w ASP.NET Core, relacyjnej bazy PostgreSQL, uwierzytelniania JWT, kontroli dostępu oraz pracy zespołowej z systemem kontroli wersji.

Efektem końcowym będzie aplikacja możliwa do uruchomienia na komputerze z Dockerem oraz posiadająca udokumentowany sposób instalacji i obsługi.

4.2 Główne założenia:

(dla każdego założenia/zadania stworzyć DoD (Definition of Done)

1. Architektura – React + ASP.NET Core + PostgreSQL + Docker Compose
   DoD: Frontend, API i baza danych działają poprawnie, a komunikacja między usługami jest prawidłowa.
2. Użytkownicy i bezpieczeństwo: rejestracja, logowanie, JWT oraz role User/Admin.
   DoD: użytkownik może utworzyć konto i się zalogować, a chronione operacje są dostępne wyłącznie dla właściwej roli.
3. Katalog i administracja: produkty i kategorie z operacjami CRUD.
   DoD: dane zapisują się w PostgreSQL, a zmiany wykonane przez administratora są widoczne w interfejsie React.
4. Koszyk i zamówienia: dodawanie/usuwanie produktów, wyliczanie wartości i złożenie zamówienia.
   DoD: zamówienie jest zapisane z pozycjami, kwotą, użytkownikiem i statusem.
5. Jakość i współpraca: Git, GitHub, Jira, podział zadań, testy krytycznych funkcji i README.
   DoD: kod znajduje się w repozytorium, główne scenariusze przechodzą testy

5. Wymagania funkcjonalne:

5.1 Podstawowe funkcjonalności (must-have):

Nazwa funkcjonalności	Kryteria akceptacji
Rejestracja, logowanie i role użytkowników	Można utworzyć konto i zalogować się. API wydaje poprawny token JWT. Endpointy administratora są niedostępne dla zwykłego użytkownika.
Katalog produktów i panel administratora	Użytkownik widzi produkty i kategorie oraz może wyszukiwać. Administrator może dodawać, edytować i usuwać produkty oraz kategorie; zmiany zapisują się w bazie.
Koszyk i obsługa zamówień	Użytkownik może dodać/usunąć produkt, zmienić ilość i złożyć zamówienie. Suma jest liczona poprawnie, zamówienie zapisuje się z pozycjami i statusem, a użytkownik widzi historię zamówień.

5.2 Opcjonalne funkcjonalności (nice-to-have):

Nazwa funkcjonalności	Kryteria akceptacji
Lista ulubionych produktów	Zalogowany użytkownik może dodać/usunąć produkt z ulubionych, a lista jest zapisana i dostępna po ponownym zalogowaniu.
Opinie i oceny produktów	Zalogowany użytkownik może dodać ocenę i opinię; średnia ocena produktu jest wyświetlana, a dane są zapisane w bazie.
Filtrowanie, sortowanie i paginacja	Katalog pozwala łączyć filtry, sortować

	wyniki i przechodzić między stronami.

6. Wymagania niefunkcjonalne:

Bezpieczeństwo: hasła przechowywane wyłącznie w postaci haszowanej; autoryzacja oparta na JWT i rolach; walidacja danych wejściowych po stronie API.

Wydajność i niezawodność: typowe operacje katalogu i koszyka powinny odpowiadać w czasie do 10 s w środowisku lokalnym; błędy API muszą zwracać czytelne kody HTTP, a integralność danych zapewniają relacje i ograniczenia PostgreSQL.

Przenośność i utrzymywalność: aplikacja uruchamiana przez Docker Compose; czytelny podział frontend/backend; kod wersjonowany w Git; README zawiera instrukcję instalacji i uruchomienia.

7. Harmonogram prac:

Data zajęć	Opis realizowanych zagadnień
5.10.2026	Analiza wymagań, wybór zakresu projektu i technologii, podział ról, przygotowanie karty projektu.
12.10.2026	Projekt architektury, model ER bazy danych, kontrakt REST API i utworzenie backlogu w Jira.
19.10.2026	Utworzenie repozytorium Git, szkieletu React i ASP.NET Core; konfiguracja PostgreSQL i Docker Compose.
26.10.2026	Modele danych, DbContext i migracje EF Core; implementacja rejestracji, logowania, JWT i ról.
2.11.2026	Implementacja backendu produktów i kategorii: CRUD, walidacja danych i podstawowy panel administratora.
9.11.2026	Implementacja interfejsu React: routing, katalog produktów, kategorie i szczegóły produktu.
16.11.2026	Integracja katalogu z API; wyszukiwanie, filtrowanie i obsługa stanów błędów/ładowania.
23.11.2026	Implementacja koszyka po stronie API i React; zmiana ilości, usuwanie pozycji, obliczanie sumy.
30.11.2026	Implementacja zamówień: tworzenie zamówienia, statusy i widok administratora.
7.12.2026	Integracja autoryzacji i panelu administratora; ograniczenie operacji według ról.
14.12.2026	Testy integracyjne głównych scenariuszy, walidacja danych, obsługa wyjątków i poprawki błędów.
21.12.2027	Refaktoryzacja, funkcje opcjonalne, dopracowanie UI.
11.01.2027	Końcowe testy całości, poprawki, dokumentacja techniczna/użytkowa i przygotowanie prezentacji.
18.01.2027	Prezentacje końcowych wersji projektów, oddanie dokumentacji.
25.01.2027	Końcowe zaliczenie projektu

8. Zarządzenie ryzykiem:

8.1 Identyfikacja ryzyk

Ryzyka: opóźnienia i zbyt szeroki zakres; problemy z integracją frontend–backend; konflikty w Git; trudności z nowymi technologiami; błędy konfiguracji PostgreSQL/Docker; nieobecność członka zespołu.

Dodatkowe ryzyka: błędy bezpieczeństwa/autoryzacji, utrata danych lub migracji oraz wykrycie krytycznych błędów dopiero pod koniec projektu.

8.2. Plan reagowania

Reakcja: najpierw realizacja funkcji must-have i cotygodniowa kontrola postępu w Jira; małe zadania i regularne commity; praca na osobnych branchach, pull requesty i wspólny kontrakt API; integracja modułów od początku, a nie dopiero na końcu.

Problemy techniczne są izolowane przez Docker i dokumentowane; baza jest odtwarzana z migracji i kopii testowych; krytyczne scenariusze są testowane na bieżąco. W razie opóźnień zadania są redystrybuowane, a funkcje nice-to-have odkładane.

9. Dokumentacja:

9.1 Dokumentacja techniczna (Doxygen)

……

9.2 Dokumentacja użytkowa

……

9.3 Dokumentacja wdrożeniowa

……

Link do Jiry:

……